

# </ Generative Adversarial Network [GAN]

(DCGAN, WGAN-GP, ProGAN)

/>

} /> [

DATS 6303 Deep Learning

Pratik Shetty  
Josh Schechter  
Jeongwon Yoo

# </ Table of contents

{01}

Overview

{02}

Architectures

{03}

Training Curves

{04}

Model Summary

{05}

Image Gallery

{06}

Conclusions



# Overview

01

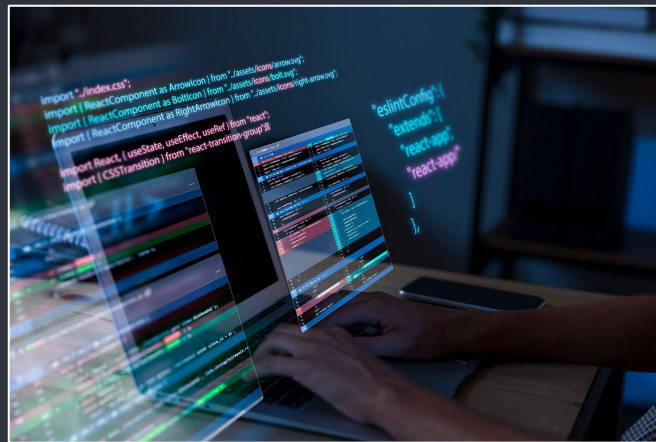


# </ Overview

Goal: Learn about the core components of GAN architectures

Experiment: Trained three GAN architectures on a dataset of real and fake images

1. DCGAN: Deep Convolutional GAN, a standard architecture for image generation.
2. WGAN-GP: Wasserstein GAN with Gradient Penalty, known for stable training.
3. ProGAN: Progressive Growing GAN, excels at generating high-resolution images.





# Architectures

02



# Shared Setup

{100}

Latent Dim

{3[RGB]}

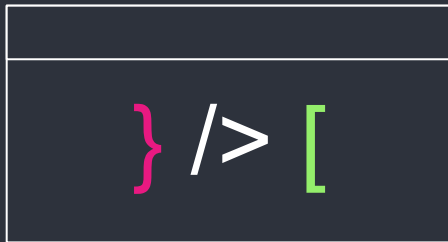
channels

{64 / 128}

Image size

{FID}

Metric, Fréchet Inception Distance



/>

1 0 1 1    0 1 1    0 1    1 0 1 1 0 0 1    1 0    1 1 0 1 1    0 1 1    0 1    1 1 0 1 1 0    1 1 0 1 1 1    1 1 0 1

# </ DCGAN

The baseline GAN model used in our project for 64×64 face generation.

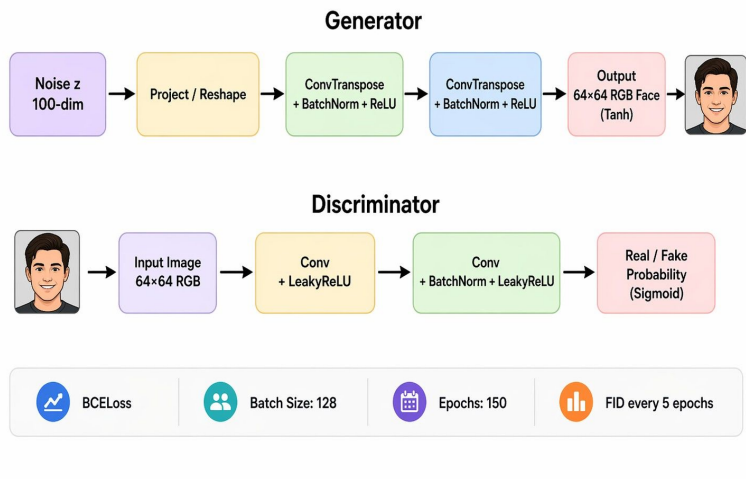
Extending the original GAN framework by replacing simple fully connected layers with deep convolutional layers in both the generator and discriminator.

- Images contain spatial structure
- Nearby pixels are related, and meaningful patterns such as edges, textures, facial contours, eyes, hair regions, and repeated local structures are all things that convolutional networks learn well.

DCGAN is a much stronger image-generation baseline than a plain fully connected GAN.

# </ What is a GAN?

## Simple DCGAN Architecture



A Generative Adversarial Network (GAN)

- Generator (G): takes random noise as input and tries to generate realistic fake images
- Discriminator (D): takes an image as input and predicts whether it is real or fake

The generator improves by learning how to fool the discriminator, while the discriminator improves by learning how to better separate generated images from real images. Over time, the generator learns to produce outputs that more closely match the real image distribution.



# </ DCGAN

## ORIGINAL GAN Objective

Original GAN (Goodfellow et al., 2014) was built with two simple neural networks:

- a generator that maps random noise to an image, and a discriminator that outputs real vs fake.
- Training alternates between updating the discriminator and generator with a minimax objective, so the generator gradually learns to produce data that matches the training distribution.

## Why DCGAN Improves a Basic GAN

A vanilla GAN often uses fully connected layers. DCGAN improves this by using image-specific design principles:

- Convolutional layers in the discriminator
- Transposed convolutional layers in the generator
- Batch Normalization for more stable optimization
- ReLU activations in the generator
- LeakyReLU activations in the discriminator
- Tanh at the generator output

The architecture is now explicitly designed to learn spatially meaningful visual features

# </ What is a Transposed Convolution?

A standard convolution is usually used to extract features and often reduce spatial resolution.

A transposed convolution does the opposite: Instead of shrinking a representation, it expands a smaller feature map into a larger one.

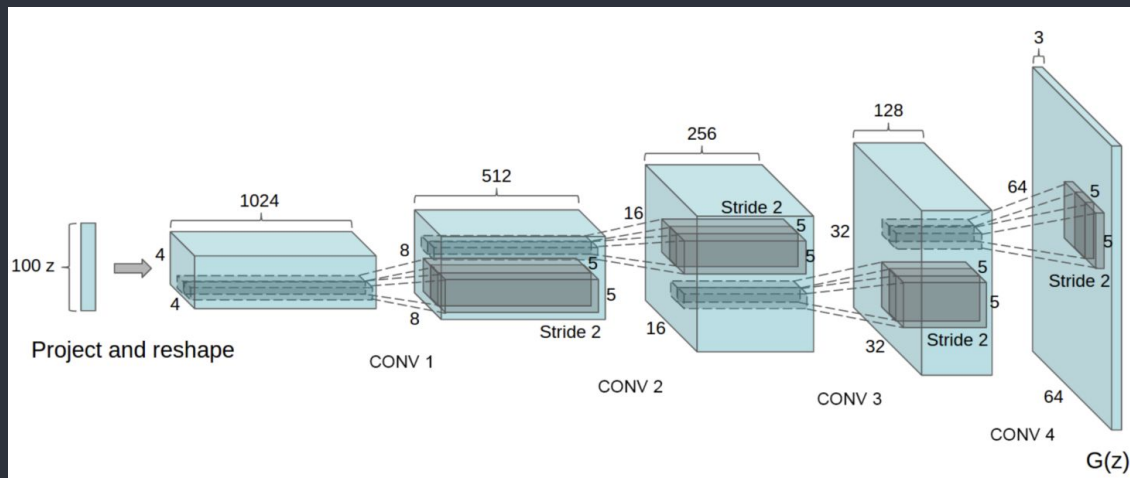
A transposed convolution learns:

- how to increase spatial resolution,
- how to place features in meaningful positions,
- how to combine low-level patterns into larger structures,
- and how to refine the image as the resolution increases.

So when we say transposed convolution is “upsampling,” we do not mean a fixed resize operation like nearest-neighbor or bilinear interpolation. We mean learned upsampling, where the network learns the filters used to create higher-resolution features.

That is why transposed convolution is central to the DCGAN generator.

# </ Generator Architecture



The generator progressively upsamples the latent representation through transposed convolution layers. Intermediate layers use BatchNorm + ReLU, and the final layer uses Tanh so that the generated output lies in the normalized range expected by the training pipeline.

# </ Discriminator Architecture

The discriminator performs the reverse process. It takes a real or generated 64×64 RGB image and progressively downsamples it through convolutional layers in order to decide whether the image is real or fake.

The discriminator uses:

- Convolutional layers for feature extraction and downsampling
- LeakyReLU activations to improve gradient flow
- BatchNorm in intermediate layers
- a final scalar output representing the probability that the input image is real

Conceptually, the discriminator learns hierarchical visual features. Early layers may respond to simple patterns such as edges and textures, while deeper layers respond to larger structures such as face shape, eyes, hair arrangement, and other image-level patterns.

1 0 1 1    0 1 1    0 1    1 0 1 1 0 0 1    1 0    1 1 0 1 1    0 1 1    0 1    1 1 0 1 1 0    1 1 0 1 1 1    1 1 0 1

# </ DCGAN Loss Functions

DCGAN uses binary cross-entropy loss. The discriminator tries to classify real images as real and generated images as fake, while the generator tries to make fake images appear real to the discriminator.

Discriminator loss:

$$L_D = -(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))])$$

- reward the discriminator for assigning high confidence to real images
- reward the discriminator for assigning low confidence to generated images

Generator loss(non-saturating form):

$$L_G = -\mathbb{E}_{z \sim p_z} [\log D(G(z))]$$

The generator is rewarded when the discriminator assigns a high “real” probability to generated images. This practical non-saturating generator loss is commonly used because it provides better gradients than the original saturating form.

# </ Why the Discriminator Loss Matters?

The discriminator loss is important because it controls how strongly the model separates real data from generated data. If the discriminator becomes too strong too early:

- it can confidently reject fake images,
- the generator may receive weak or unstable gradients,
- learning can slow down,
- or the adversarial game can become unstable.

This is one of the core reasons GAN training is difficult. The balance between generator and discriminator is delicate. If one side becomes too dominant, the other side can stop learning effectively.

This matters a lot for understanding later GAN variants. In particular, WGAN and WGAN-GP were introduced partly to improve the quality of gradients and make training more stable than the original probability-based discriminator formulation used in DCGAN.

# </ WGAN-GP

WGAN-GP is an improved version of the Wasserstein GAN that incorporates a gradient penalty to enforce the Lipschitz constraint. This architecture helps stabilize training and allows for better convergence, resulting in higher quality generated images.

## How WGAN-GP Works

1. The generator maps noise  $z$  to synthetic images  $G(z)$ .
2. The critic (formerly discriminator) scores real images  $x$  and generated images  $G(z)$  with scalar values.
3. Training maximizes the critic gap between real and fake scores.
4. The generator is trained to increase critic scores on fake samples.
5. A gradient penalty term enforces a 1-Lipschitz critic, which stabilizes training.

# </ Wasserstein Objective

The critic approximates the **Earth Mover** (Wasserstein-1) distance

- “The distance between the real and sampled images”

$$\mathcal{L}_C = \mathbb{E}_{\tilde{x} \sim P_g} [D(\tilde{x})] - \mathbb{E}_{x \sim P_r} [D(x)]$$

$$\mathcal{L}_G = - \mathbb{E}_{z \sim p(z)} [D(G(z))]$$



# </ Earth Mover Distance

## Why It helped Beyond DCGAN

Earth Mover Distance: Why It Helped Beyond DCGAN

1. Before Earth Move: BCE loss
  - Binary classification by the discriminator: Probability of Real or Fake
  - When the probability distributions for the real and fake samples have little overlap, the discriminator easily classifies samples
    - This means the information to the gradient isn't helpful
      1. Basically, tells generator that the images are bad, not why they are bad
- Consequences are mode collapse, unstable training dynamics, and lack of ability to train on higher resolutions.

## 2. Solution: Earth Mover Distance

- WGAN proposed using the Earth Mover (Wasserstein-1) distance as a loss function
  - Scalar value that tells the difference between real and fake, not just a binary “good or bad”
- With a scalar distance value, the information returned to generator should be much more useful, leading to smoother training dynamics
  - The scalar output is a distance value, that basically tells how far apart the real and fake distributions are
- The **constraint** is that the output must be 1-Lipschitz
  - 1-Lipschitz means that the output of the critic must increase at most the same as the input. This ensures that the critic is smooth and returns meaningful gradients.
  - Originally, the WGAN paper addressed this with **weight clipping**, but they said themselves this was a terrible idea and encouraged further research

# </ Problem with Weight Clipping [bounds on weights]

1. If the weights are clipped too small, the critic is too constrained (gradients vanish as nothing useful is being found)
2. If weights are clipped too large, the critic has more leeway than needed, and can diverge (gradients explode as critic is finding too much)

# </ Gradient Penalty [A good solution]

## How is gradient penalty computed?

1. Interpolate between real and fake samples
2. Compute the gradient of the critic output with respect to these interpolated samples.
3. The gradient penalty is the squared difference between the norm of this gradient and 1, scaled by a penalty coefficient ( $\lambda$ ).

## What is the result?

The gradient penalty enforces the Lipschitz constraint without weight clipping, which can lead to better convergence and higher quality images. The gradient penalty is computed as follows:

$$\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}, \quad \epsilon \sim U(0,1)$$

$$\mathcal{L}_{GP} = \lambda \mathbb{E}_{\hat{x}} (\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2$$

**Final WGAN-GP Critic Loss Function:**

$$\mathcal{L}_C = \mathbb{E}_{\tilde{x} \sim P_{\tilde{x}}} [D(\tilde{x})] - \mathbb{E}_{x \sim P_r} [D(x)] + \mathcal{L}_{GP}$$

# </ Gradient Penalty

## Gradient Penalty Benefits

1. Enforces the Lipschitz constraint more effectively than weight clipping, leading to stable convergence
2. Allows training on higher resolution images without mode collapse or instability

# </ PROGAN

Progressively grows the generator and discriminator networks during training.

- the model to learn coarse features first and then gradually refine them, leading to improved stability and higher quality generated images.

## How ProGAN Works

1. Training starts at a very low resolution(4×4) for both the generator and discriminator.
2. New layers are added progressively to double the resolution at each step
3. Each new layer is introduced gradually using a fade-in mechanism controlled by an alpha parameter (0→1).
4. The generator maps noise  $z$  to synthetic images, and the discriminator scores real vs. fake images at each resolution.
5. A gradient penalty is applied to stabilize training.

1 0 1 1   0 1 1   0 1   1 0 1 1 0 0 1   1 0   1 1 0 1 1   0 1 1   0 1   1 1 0 1 1 0   1 1 0 1 1 1   1 1 0 1

# </ Fade-In Mechanism

When a new layer is added, it is blended in smoothly using an alpha parameter:

$$\text{output} = \alpha \cdot \text{new\_layer} + (1 - \alpha) \cdot \text{upsampled\_previous}$$

- $\alpha = 0$ : only the previous resolution output is used
- $\alpha = 1$ : fully switched to the new higher resolution layer
- This prevents sudden shocks to already-learned features when adding new layers.

1 0 1 1   0 1 1   0 1   1 0 1 1 0 0 1   1 0   1 1 0 1 1   0 1 1   0 1   1 1 0 1 1 0   1 1 0 1 1 1   1 1 0 1

# </ PROGAN

## Key Techniques

- 1. Equalized Learning Rate**
  - dynamically scales weights so all layers train at equal speed
- 2. Pixelwise Feature Normalization**
  - normalizes pixel features in Generator to prevent signal explosion
- 3. Minibatch Std Dev**
  - adds diversity stats to Discriminator to prevent mode collapse

## Why Progressive Training Helps

- Learns global structure first (shape, layout), then fine details (textures, hair)
- Keeps training balanced – Discriminator sees progressively harder tasks
- More stable training and better image quality than full-resolution from scratch
- Downside: Higher complexity and careful tuning required (fade-in, learning rate)



</>

# Training Curves

03

} /> [

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 0 1

# </ Training Curves

## Fréchet Inception Distance [FID]

The FID score is a widely used metric for evaluating the quality of images generated by GANs. It is a distance measurement between the distribution of generated images and the distribution of real images. A lower FID score indicates that the generated images are more similar to the real images, while a higher FID score indicates that the generated images are less similar to the real images.

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr} \left( \Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2} \right)$$

Where:

$\mu_r, \mu_g$  – mean feature vectors of real and generated images

$\Sigma_r, \Sigma_g$  – covariance matrices of real and generated image features

Tr – trace of a matrix

| Qualities                  | IS  | FID |
|----------------------------|-----|-----|
| Evaluates real images      | No  | Yes |
| Accounts for mode collapse | Yes | Yes |
| Requires labels            | Yes | No  |

64

Showing available training curves.

The graph displays the training progress of a GAN. The discriminator loss (dark blue) starts at approximately 0.4, drops to 0.3, and then rises to 1.0 at epoch 20. It remains relatively stable, fluctuating between 0.5 and 1.0 for the rest of the training. The generator loss (light blue) starts at approximately 1.3, drops to 0.3 at epoch 20, and then rises sharply to approximately 3.5 by epoch 150, indicating a significant increase in the generator's loss over time.

The graph illustrates the trend of COVID-19 cases in the Netherlands. It starts with a high number of cases (around 450) in March 2020, followed by a period of fluctuation and a significant drop to about 180 cases by May 2020. From there, the number of cases continues to decline steadily, reaching a low point of approximately 20 cases in January 2021, and then remains relatively stable at a low level through March 2021.

| Month          | Number of Cases (approx.) |
|----------------|---------------------------|
| March 2020     | 450                       |
| April 2020     | 420                       |
| May 2020       | 180                       |
| June 2020      | 100                       |
| July 2020      | 80                        |
| August 2020    | 70                        |
| September 2020 | 50                        |
| October 2020   | 40                        |
| November 2020  | 30                        |
| December 2020  | 25                        |
| January 2021   | 20                        |
| February 2021  | 25                        |
| March 2021     | 20                        |

Showing available training curves.

| Epoch | critic_loss | generator_loss |
|-------|-------------|----------------|
| 0     | -25         | -25            |
| 20    | -10         | -10            |
| 40    | -5          | 0              |
| 60    | -4          | 2              |
| 80    | -4          | 3              |
| 100   | -4          | 4              |
| 120   | -4          | 5              |
| 140   | -4          | 6              |
| 160   | -4          | 7              |
| 180   | -4          | 8              |
| 200   | -4          | 9              |
| 220   | -4          | 10             |
| 240   | -4          | 10             |
| 260   | -4          | 10             |

The line graph illustrates the trend of deaths per 100,000 population in the United States over a century. The y-axis represents the death rate, ranging from 0 to 120. The x-axis represents the year, from 1900 to 2020. The death rate starts at approximately 115 in 1900, drops sharply to about 65 by 1910, and then fluctuates between 40 and 50 until the mid-1920s. After 1925, there is a steady decline, reaching approximately 20 by 2020.

| Year | Deaths per 100,000 population |
|------|-------------------------------|
| 1900 | 115                           |
| 1905 | 75                            |
| 1910 | 65                            |
| 1915 | 55                            |
| 1918 | 50                            |
| 1919 | 45                            |
| 1920 | 48                            |
| 1925 | 48                            |
| 1930 | 38                            |
| 1935 | 38                            |
| 1940 | 32                            |
| 1945 | 30                            |
| 1950 | 28                            |
| 1955 | 27                            |
| 1960 | 26                            |
| 1965 | 25                            |
| 1970 | 24                            |
| 1975 | 23                            |
| 1980 | 22                            |
| 1985 | 23                            |
| 1990 | 22                            |
| 1995 | 22                            |
| 2000 | 22                            |
| 2005 | 22                            |
| 2010 | 22                            |
| 2015 | 21                            |
| 2020 | 20                            |

Showing available training curves.

The graph displays the training progress of a GAN. The x-axis represents iterations from 0 to 1600. The y-axis represents the loss value from -20 to 100. The discriminator loss (dark blue line) stays consistently low, near zero, indicating it is effectively distinguishing between real and generated data. The generator loss (light blue line) starts around 10, fluctuates, and then begins a sharp ascent after iteration 1000, reaching a value of approximately 100 by iteration 1600. This suggests the generator is struggling to produce realistic samples as training progresses.

The line graph illustrates the projected number of deaths per 100,000 population in the United States from 2000 to 2030. The y-axis represents the death rate, ranging from 0 to 60 in increments of 10. The x-axis represents the year, from 2000 to 2030 in increments of 2. The data shows a steady decline from 2000 to 2002, followed by a period of relative stability with minor fluctuations between 2003 and 2015. After 2015, the rate continues to decline more gradually, reaching approximately 42 deaths per 100,000 by 2030.

| Year | Deaths per 100,000 |
|------|--------------------|
| 2000 | 60                 |
| 2001 | 58                 |
| 2002 | 50                 |
| 2003 | 49                 |
| 2004 | 48                 |
| 2005 | 49                 |
| 2006 | 47                 |
| 2007 | 45                 |
| 2008 | 44                 |
| 2009 | 45                 |
| 2010 | 44                 |
| 2011 | 45                 |
| 2012 | 44                 |
| 2013 | 44                 |
| 2014 | 44                 |
| 2015 | 41                 |
| 2016 | 43                 |
| 2017 | 42                 |
| 2018 | 41                 |
| 2019 | 40                 |
| 2020 | 42                 |
| 2021 | 43                 |
| 2022 | 43                 |
| 2023 | 43                 |
| 2024 | 41                 |
| 2025 | 42                 |
| 2026 | 42                 |
| 2027 | 41                 |
| 2028 | 41                 |
| 2029 | 42                 |
| 2030 | 43                 |

1 0 1 1    0 1 1    0 1    1 0 1 1 0 0 1    1 0    1 1 0 1 1    0 1 1    0 1    1 1 0 1 1 0    1 1 0 1 1 1    1 1 0 1

Select Image Size

128

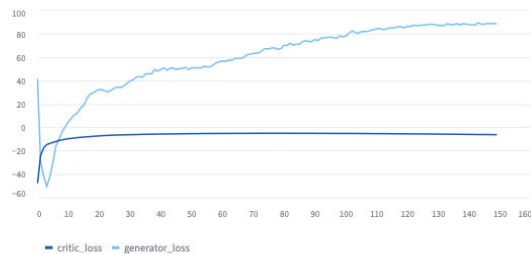
## DCGAN

Placeholder: DCGAN training logs for 128x128 are not available yet.

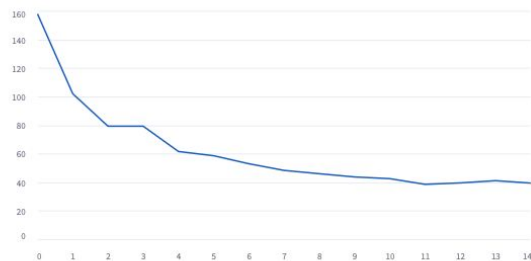
## WGAN-GP

Showing available training curves.

### Loss Curves



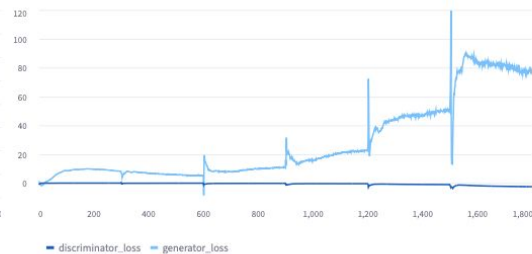
### FID Score



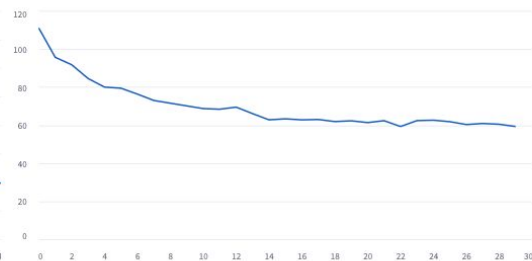
## ProGAN

Showing available training curves.

### Loss Curves



### FID Score



1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1

</>

# Model Summary

04

} /> [

1 0 1 1   0 1 1   0 1   1 0 1 1 0 0 1   1 0   1 1 0 1 1   0 1 1   0 1   1 1 0 1 1 0   1 1 0 1 1 1   1 1 0 1

Table of Contents

Go to  
Model Summary

Generative Adversarial Network (GAN) Dashboard

Model Summaries and Comparison

Compare all GAN models using training metrics and generated samples.

Image Size  
64

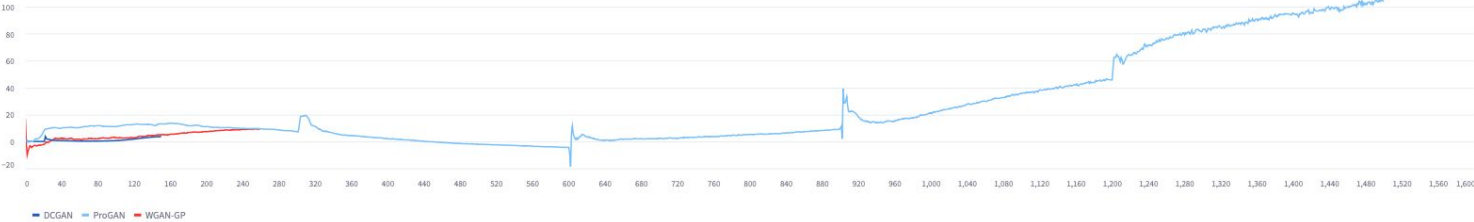
Models  
WGAN-GP x DCGAN x ProGAN x

Comparison Metric  
Generator Loss  
FID  
Generator Loss  
Discriminator/Critic Loss

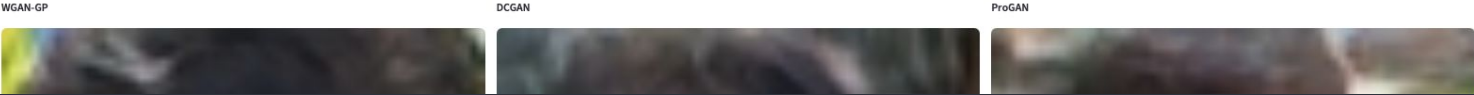
Summary

|   | Model   | Status    | Epochs | Latest FID | Best FID | Final Generator Loss | Final Disc/Critic Loss | Sample Available         |
|---|---------|-----------|--------|------------|----------|----------------------|------------------------|--------------------------|
| 0 | WGAN-GP | Available | 259    | 21.287     | 21.2481  | 9.3579               | -2.7415                | <input type="checkbox"/> |
| 1 | DCGAN   | Available | 150    | 24.2553    | 22.8093  | 3.5812               | 0.5833                 | <input type="checkbox"/> |
| 2 | ProGAN  | Available | 1500   | 41.9365    | 39.527   | 104.6059             | -1.898                 | <input type="checkbox"/> |

Generator Loss Across Models



Generated Sample Comparison



</>

# Image Gallery

05

} /> [

1 0 1 1   0 1 1   0 1   1 0 1 1 0 0 1   1 0   1 1 0 1 1   0 1 1   0 1   1 1 0 1 1 0   1 1 0 1 1 1   1 1 0 1

## Table of Contents

Go to

Image Gallery

# Generative Adversarial Network (GAN) Dashboard

## Generated Images

View images generated by the GANs from generation script.

Model

wgan\_gp

wgan\_gp

drgan

progan

Image Size

64



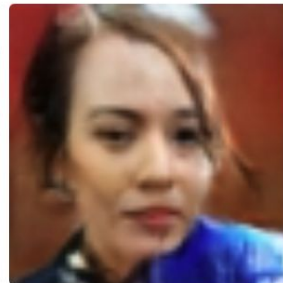
009990.png



009991.png



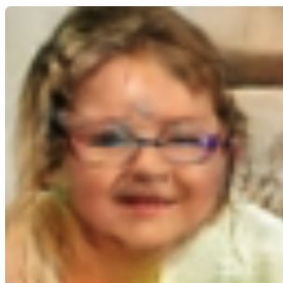
009992.png



009993.png



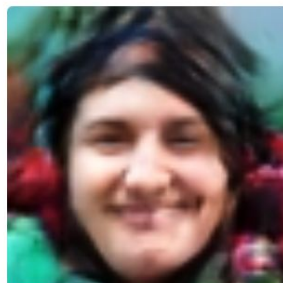
009994.png



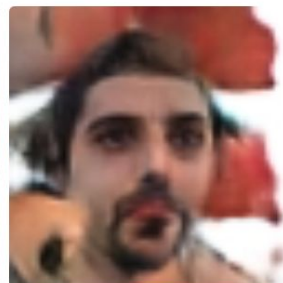
009995.png



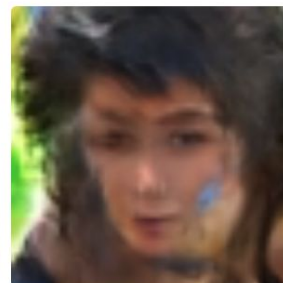
009996.png



009997.png



009998.png



009999.png

1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 1 0 1



</>

# Conclusions

06

} /> [

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1

# </ Conclusions and Findings

## Key Results

- WGAN-GP delivered the best overall model performance in this project. It also gave the most stable training behavior across runs.
- DCGAN remains a useful baseline and is kept in our final comparison.
- ProGAN was slower to train, and with our allotted time we could not fully tune it for larger resolutions. In theory, it has the potential to outperform WGAN-GP on higher resolution images, as it is designed to find the details first and work its way up.

## Observed Training Dynamics

- Generator and discriminator balance was critical; imbalance led to unstable learning.
- Training became harder as image resolution increased, requiring stronger regularization and tuning.

# </ Model-by-Model Takeaways

## DCGAN

- Good baseline architecture and easiest to compare against.
- Kept in the project as a reference model.

## WGAN-GP

- Best overall performer for this dataset and setup.
- Most reliable optimization behavior and smoothest training progression.

## ProGAN

- Training was noticeably slower than our other models.
- Needed more fine tuning time than we had, especially for larger resolutions.
- The benefits of ProGAN are best seen at higher resolutions, which require more time.



# </ Limitations and Future Work

## Limitations

- Limited training budget and time constrained model tuning.
- Evaluation focused on a narrow set of metrics and visual checks.
- Results are specific to our dataset, however we encourage applying this methodology to other datasets and domains.

## Future Work

- Apply methodology to other datasets that are more complex or diverse.
- Explore additional architectures (e.g., StyleGAN).
- Conduct more extensive hyperparameter tuning, especially for ProGAN (need weeks to fully optimize).
- Evaluate on more diverse datasets, try training on higher resolutions and different domains.



# </ Thanks!

Do you have any questions?



/>

} /> [

**CREDITS:** This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)

Please keep this slide for attribution